

Predictive Safety Control for Human- Robot Collaboration in a PyBullet Pick-and-Place Workspace

Final Project Report

Student name: Yifan Wang

Student ID: 200102494

Module / Programme: CS3IP

Supervisor: Dr Zhuangzhuang Dai

Word count: 8425

Abstract

Safety remains a major barrier to effective human-robot collaboration in flexible manufacturing environments. Traditional industrial safety methods often rely on physical cages or conservative emergency stops, which reduce workspace efficiency and limit collaboration. This project investigates whether **predictive obstacle tracking** can improve safety without making the robot overly conservative.

A real-time safety control framework was developed for a Franka Emika Panda robot in a simplified PyBullet pick-and-place workspace. The system combines 3D point-cloud perception, short-horizon constant-velocity Kalman prediction, and a modified Artificial Potential Field (APF) controller, with axis-aligned bounding box (AABB) perception and payload-aware boundary adaptation so collision checking covers both the arm and the grasped object.

The framework was evaluated through a 150-trial velocity-sensitivity Monte Carlo benchmark across five obstacle-speed conditions, including an “Insane” setting of 0.017 m per simulation step (4.08 m/s at 240 Hz). A trial failed if any manipulator-payload collision occurred, final placement exceeded tolerance, or completion time exceeded 60 s. Under this strict zero-collision criterion, success declined from 93.3% at Slow and Medium speeds to 70.0% at Insane, giving an **overall mean of 85.3%**. Across successful zero-collision trials, mean minimum clearance stayed within 0.150–0.166 m, with the lowest value at Insane. A separate 150-trial narrow-passage benchmark achieved **96.7% success with zero collisions**. A latency ablation against a reactive APF-only baseline showed a speed-dependent cross-over: prediction improved clearance at low-to-moderate speeds, while the reactive baseline achieved larger clearance at the highest speeds. The predictor is therefore interpreted as useful within a **bounded operating envelope**, not as a uniform improvement across all safety and smoothness metrics.

Overall, the results support AABB-aware, payload-conscious avoidance as the core safety mechanism, with short-horizon prediction acting as an auxiliary layer whose benefit depends on obstacle speed, prediction horizon, and sim-to-real limits at high contact speeds.

1. Introduction

1.1 Project Background and Motivation

Modern manufacturing is moving towards flexible human-robot collaboration rather than fully separated automation. In these shared workspaces, industrial robots can no longer rely only on fixed cages or simple stop-on-contact rules. Although such measures improve safety, they also reduce workspace flexibility and make close

cooperation inefficient.

For collaboration to be practical, the robot must respond to nearby human motion early enough to avoid unnecessary stopping while still **maintaining a safe distance**. This is especially important in dynamic factory tasks, where a worker may enter the robot workspace unexpectedly during handling or transfer operations.

This project develops and evaluates a predictive safety control framework in a PyBullet simulation of a factory pick-and-place workspace. Rather than removing safety constraints, it aims to enforce them earlier by anticipating short-term obstacle motion and adjusting avoidance before proximity becomes critical, improving the balance between safety and task continuity in human-robot collaboration (HRC).

This project is therefore best understood as an applied software-engineering research project. The main contribution is not only the implementation of a PyBullet-based robotic control system, but also the experimental evaluation of a predictive, payload-aware avoidance framework under controlled benchmark conditions. To reflect this, the report links the software architecture to the benchmark design, showing how the system was built and how its safety and performance claims were tested reproducibly.

1.2 Problem Definition and Challenges

A central challenge in human-robot collaboration is that human motion is dynamic, irregular, and difficult to model perfectly in real time. In a shared workspace, a robot often has only **partial and delayed observations** of a moving person or limb (Flacco et al., 2012). If the controller reacts too late, contact risk increases. If it reacts too conservatively, the task becomes inefficient or may stop unnecessarily.

This problem is particularly visible in reactive obstacle-avoidance methods such as the Artificial Potential Field (APF), a classical real-time obstacle-avoidance approach introduced for manipulators and mobile robots by Khatib (1986). APF is attractive because it is simple and fast, but prior work has shown that it can suffer from **local minima, oscillatory behaviour, and unstable responses** around moving obstacles (Koren and Borenstein, 1991). In practical terms, this can lead to abrupt motion, repeated hesitation, or emergency-stop-like behaviour when the workspace becomes crowded.

The project also faces an additional engineering challenge beyond basic obstacle avoidance. During material handling, the effective collision volume is not limited to the robot arm itself; it also includes the grasped object. A safety mechanism that ignores the payload geometry may underestimate the true occupied space and therefore weaken collision protection in realistic task scenarios.

Based on these issues, the project asks the following question: can a lightweight

predictive model improve the safety and smoothness of APF-based avoidance in dynamic HRC settings, while remaining fast enough for a high-frequency control loop?

1.3 Research Aim and Key Contributions

The project aims to design and evaluate a predictive safety framework for dynamic obstacle avoidance in HRC. The problem is not only reacting to nearby obstacles, but maintaining a usable motion estimate under delayed, noisy observations and converting that estimate into a **real-time safety response during payload handling**. The work therefore examines whether a **lightweight Kalman prediction module** can improve a reactive APF controller under realistic sensing and timing constraints. It makes two main contributions.

1.3.1 Predictive State Estimation via a 6D Linear Kalman Filter

The first contribution is the integration of a **6D linear Kalman filter** into the avoidance pipeline to estimate obstacle position and velocity in real time. The predictor maintains a short-horizon estimate of obstacle motion and adjusts its effective prediction horizon according to simple motion trends such as approaching or leaving.

This design was chosen because the project needed a method that was computationally lightweight, deterministic, and suitable for a high-frequency loop. The filter is not intended to model complex human intent in a semantic sense; it reduces the effect of sensing delay and observation noise so that avoidance can begin earlier and more smoothly. Its quantitative contribution is evaluated through the latency ablation in Section 6.2.6.

1.3.2 Integrated Real-Time Perception, Avoidance, and Payload-Aware Safety Checking

The second contribution is the implementation of a complete real-time pipeline that links perception, prediction, avoidance, and safety checking within a single simulated framework. A 3D point-cloud process is used to reconstruct obstacle observations, which are then passed to the Kalman filter (KF) and the dynamically modulated APF controller.

The APF component was adapted so that repulsive behaviour could respond not only to current obstacle proximity but also to predicted motion trends. This allows the robot to begin evasive motion earlier than a purely reactive controller, particularly when the obstacle is moving quickly across the workspace.

A further engineering contribution is the **payload-aware collision update**. After

grasping, the system expands its collision boundary using the geometry of the carried object rather than treating the manipulator alone as the protected body. This is a practical addition aimed at improving safety coverage during handling tasks, where the payload may become the first point of contact.

2. Literature Review and Technical Background

2.1 Critical Review of Predictive Avoidance Architectures

In vision-based HRC, obstacle observations are often noisy and slower than the control loop. Depth-based collision avoidance and workspace-aware safety control are established challenges (Flacco et al., 2012), so predictive systems often use state estimators to maintain **obstacle motion estimates between sensor updates**. Extended Kalman Filter (EKF) and Particle Filter (PF) methods can model more complex motion than a linear model, but this project required reliable prediction within a strict real-time control budget rather than maximum modelling flexibility.

The EKF extends Kalman filtering to nonlinear systems through local linearisation, which is useful for strongly nonlinear dynamics. Here, however, visual updates are slower than the 240 Hz control loop, and the available measurements are geometric centroid observations rather than rich human-pose data. Repeated Jacobian computation would add timing and implementation overhead without a proportionate safety benefit.

Particle Filters provide even greater flexibility for non-Gaussian and multimodal motion estimation (Arulampalam et al., 2002). Their main drawback in this context is computational cost. Maintaining robust tracking typically requires a sufficiently large particle set, together with repeated resampling, which makes execution time less predictable. For a safety-oriented controller running at 240 Hz, this variability is a practical disadvantage. The issue in this project is not whether PF can model motion better in principle, but whether it can do so while preserving deterministic update timing.

For these reasons, the project adopts a 6D linear constant-velocity Kalman Filter. Although the model is less expressive under abrupt accelerations, it is computationally lightweight, deterministic, and appropriate for short-horizon prediction between sparse visual observations.

2.2 Comparative Analysis of Alternative Motion Planning Strategies

After estimating obstacle motion, the next question is how to use that estimate

for local avoidance. The planning goal here is not global path optimality, but **safe and smooth local response under slower visual updates**. Recent APF variants also address dynamic HRC; for example, Shang et al. (2025) combine sampling, hazard indices, and vision-based prediction. This project instead prioritises a simpler deterministic APF framework supported by short-horizon state prediction.

Sampling-based planners such as Rapidly-exploring Random Trees (RRT and RRT*) are effective for global path generation and can reduce the risk of local minima in cluttered static environments (Karaman and Frazzoli, 2011). However, this project focuses on repeated short-horizon avoidance inside an already constrained manipulation task, rather than one-off global route search. In this setting, the stochastic latency of repeated sampling and the need for additional path smoothing make RRT-style approaches harder to justify within a 240 Hz control architecture.

Alternative dynamic planning families, such as the Dynamic Window Approach (DWA) in mobile robotics and Model Predictive Control (MPC) in constrained control, handle feasibility and future trajectory constraints more explicitly, but require search or optimisation at each control step (Fox et al., 1997; Mayne et al., 2000). In this project, that cost would reduce timing margin for point-cloud processing, prediction, and collision handling. Similarly, the safety-aware replanning method of Tonola et al. (2025) reports efficiency gains of up to 60%, but uses a heavier online replanning structure than the **lightweight APF+KF architecture** adopted here.

Overall, APF+KF is a practical engineering trade-off: APF provides fast local avoidance, while the Kalman filter maintains a short-horizon obstacle estimate during sparse or delayed visual updates. This improves responsiveness without exceeding the timing constraints of the control system, making the architecture suitable for repeatable simulation-based evaluation.

2.3 Technical Approach and Platform Selection

PyBullet was selected as the main development platform because it supports fast iterative testing of perception, prediction, and control within a single simulation environment (Coumans and Bai, 2016-2021). For this project, the platform needed to do more than display robot motion: it had to support repeated collision checking, high-frequency control updates, and controlled evaluation under different obstacle behaviours. PyBullet provided a practical balance between simulation speed, scripting flexibility, and ease of integration with the Python-based control pipeline.

Another reason for using PyBullet was that the project followed a **simulation-first workflow**. The goal at this stage was not to claim direct real-world equivalence, but to build and evaluate the control logic under repeatable conditions before considering physical deployment. This made it possible to test the framework across repeated

Monte Carlo trials, compare behaviour across multiple obstacle-speed regimes, and examine how the controller responded when visual updates were relatively sparse.

The platform was therefore chosen for methodological as well as technical reasons. It allowed the project to develop the system incrementally, observe failure modes in a controlled setting, and evaluate the interaction between obstacle prediction and avoidance behaviour without the practical risks of early hardware testing.

3. Project Development Process

3.1 Development Lifecycle

The project followed an incremental development process. A stable pick-and-place baseline was built first, followed by APF-based obstacle avoidance, a 6D linear Kalman filter after the reactive layer's limitations became clear, and finally payload-aware collision-boundary adaptation once grasping and transport were stable. This staged approach helped isolate failure modes and tune subsystems before full integration.

3.1.1 Project Planning and Time Management

Project planning translated the development lifecycle into practical checkpoints for scope control, testing, and evidence collection. Each stage acted as a decision point: if the current subsystem was stable enough for repeatable tests, the project moved forward; if not, time was kept for tuning before integration. This made progress easier to monitor and helped distinguish necessary system work from optional additions.

During execution, the schedule had to be adjusted because perception tuning, Q/R calibration, and Monte Carlo benchmark runs required more time than expected. The final phase therefore prioritised reproducible evidence over adding extra features: effort was concentrated on stabilising the dual-camera pipeline, validating the 240 Hz/10 Hz multi-rate loop, collecting velocity-sensitivity and narrow-passage results, and documenting limitations. This time-management decision helped keep the project aligned with its core research question while leaving enough time to report the mixed benchmark outcomes and the limitations of the final system.

3.2 Simulation Environment and Hardware Rationale

A simulation-first workflow supported rapid testing under controlled industrial-style constraints. PyBullet was selected because its Bullet Physics engine provides real-time collision detection at 240 Hz, allowing control behaviour to be evaluated under simulated sensor latency and disturbance while maintaining stable repeatable experiments.

The Franka Emika Panda was selected because its seven-degree-of-freedom (7-DOF) kinematics provide redundancy for avoidance without disrupting the main end-effector task. To avoid relying on simulator ground-truth obstacle coordinates, the system used a **synthetic vision layer**. A top-down camera provided planar localisation, while a side-view camera estimated clearance height; together, they gave a lightweight 3D obstacle estimate without using hidden simulator state.

3.3 Risk Management and Mitigation Strategy

Risk management is important in this project due to the unpredictability of dynamic simulations and complex mathematical models. A risk assessment was carried out at the start of the project to identify **technical, algorithmic, and scheduling risks**.

A Risk Matrix (Table 3.3) was created to evaluate and address these uncertainties. Each risk was assessed based on its likelihood (Low/Medium/High) and its impact on the project (Low/Medium/High), followed by a corresponding mitigation strategy.

Table 3.3: Project Risk Assessment and Mitigation Matrix

ID	Risk Description & Category	Likelihood	Impact	Mitigation Strategy
R1	Physics engine instability (Technical): gripping failure or payload drop during fast evasive motion.	Medium	High	Dynamic descent and payload-aware gripping; tune friction coefficients; use fixed constraints in early tests.
R2	Kalman Filter Divergence (Algorithmic): Improper tuning of the process noise (Q) and measurement noise (R) matrices may lead to inaccurate state predictions.	High	High	Adopt a highly iterative, data-driven tuning approach. Before full integration, use a dedicated Monte Carlo benchmarking script to tune and validate filter performance across different speed levels.
R3	APF Local Minima (Algorithmic): The Artificial Potential Field may trap the robotic arm in local minima or kinematic singularities, preventing it from reaching the target.	Medium	High	Introduce a hierarchical decision tree. If the arm is trapped, trigger a lifting recovery manoeuvre to escape the localized repulsive field.

R4	Computational Bottleneck (Performance): Real-time processing of point clouds coupled with continuous APF recalculations may cause frame-rate drops, disrupting real-time control.	Low	Medium	Optimize algorithmic complexity by vectorizing matrix operations using NumPy. Downsample the camera resolution if computational latency exceeds the 1/240 second simulation step threshold.
R5	Schedule Overrun (Management): The complexity of integrating vision, prediction, and control modules may exceed the allocated timeframe.	Medium	Medium	Prioritise a basic static APF controller as the minimum viable product (MVP), then sequentially integrate the Kalman Filter and payload-awareness modules so that a functional baseline is maintained throughout development.

The mitigation strategies reflect an iterative engineering approach that combines variable isolation, fallback mechanisms, and empirical benchmarking rather than relying solely on theoretical assumptions. This made integration failures easier to diagnose and reduced the risk that tuning issues in one module would be mistaken for system-wide failure.

4. System Design and Technical Infrastructure

4.1 Object-Oriented System Architecture

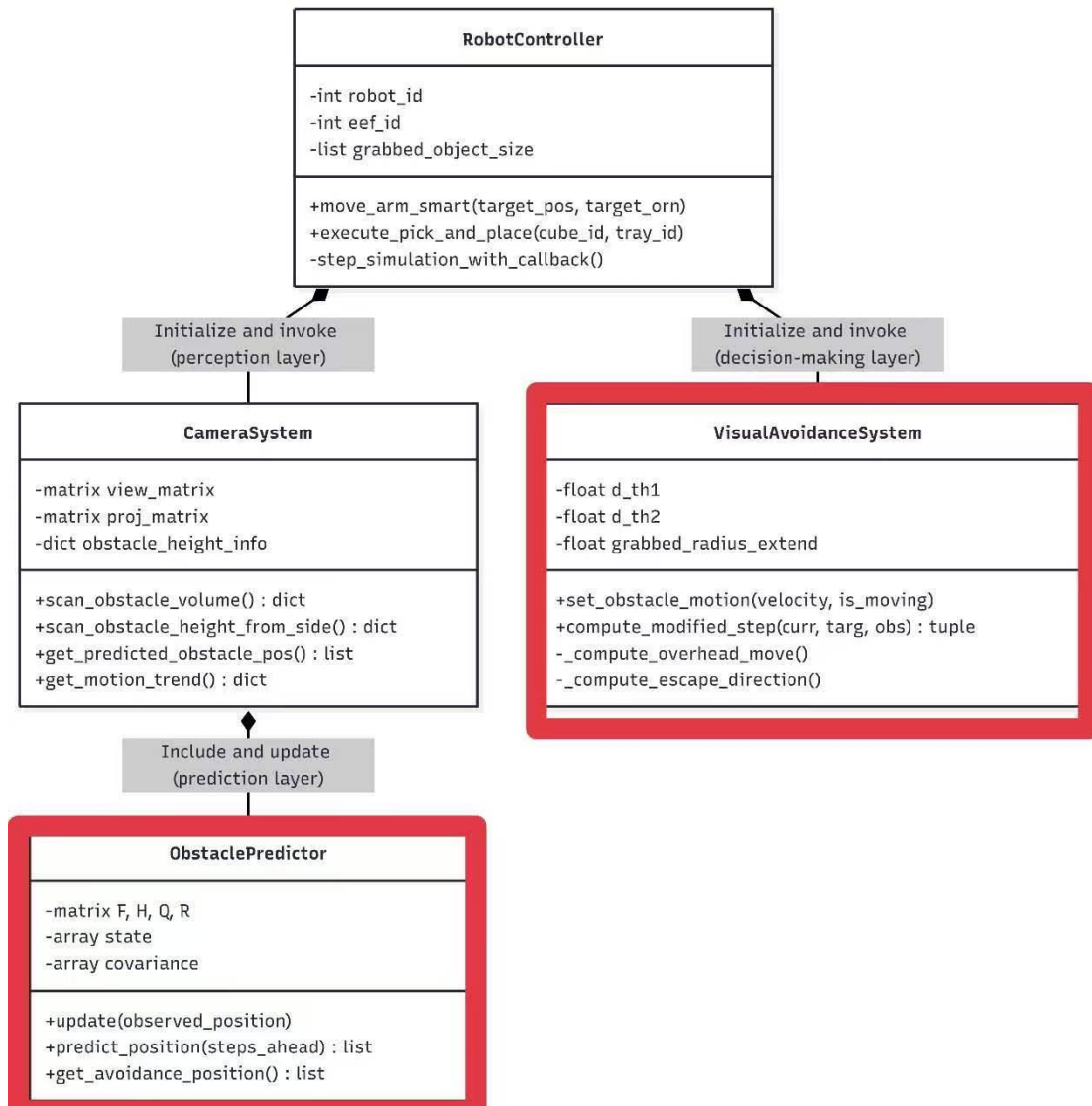


Figure 4.1: UML Class Diagram of the Control Framework

The control framework uses a **decoupled, object-oriented architecture** to support maintainability and scalability (see Figure 4.1). The **RobotController** serves as the central component, assigning tasks to independent modules. The perception layer is implemented in the **CameraSystem**, which handles 3D point cloud reconstruction and spatial awareness. The **ObstaclePredictor** is part of the perception layer and processes raw volumetric data using the 6D Linear Kalman Filter. The decision-making layer is implemented in the **VisualAvoidanceSystem**, which calculates dynamically modulated APF vectors and updates payload-aware collision boundaries. This separation allows prediction behaviour to be tuned without rewriting the kinematic execution loop.

4.2 Multi-Rate Real-Time Execution Flow

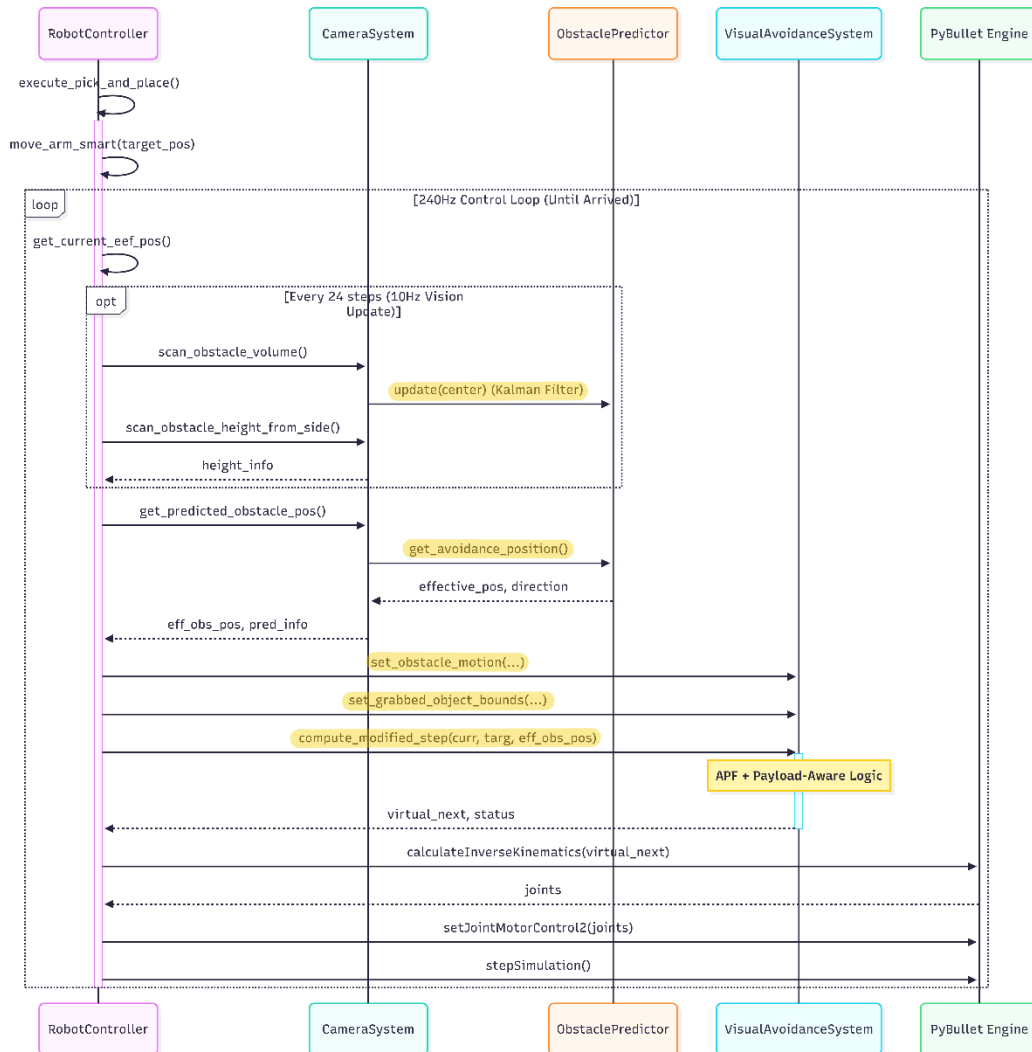


Figure 4.2: Sequence Diagram of the 240 Hz Control Loop

A key challenge in this project was handling the computational cost of 3D vision while maintaining real-time performance. As shown in the sequence diagram (Figure 4.2), this is handled using a **multi-rate execution architecture**. The main motion loop and the PyBullet physics engine run at **240 Hz (4.16 ms per cycle)** to maintain kinematic stability. More computationally expensive operations, including obstacle-volume scanning and side-view height estimation, run at a lower frequency of **10 Hz (every 24 steps)**.

The ObstaclePredictor bridges this gap between the two update rates. It updates its state matrix using 10 Hz visual observations while providing trajectory predictions to the VisualAvoidanceSystem at the full 240 Hz rate. This asynchronous data flow keeps the **high-frequency avoidance loop** supplied with the latest available estimate between camera renders, reducing delay-related blind spots.

4.3 Verification and Quality Assurance Strategy

Verification used **automated component- and system-level scripts** rather than a single end-to-end run. At component level, the 6D Kalman filter was tested under Gaussian observation noise and several Q/R settings on dynamic-obstacle traces collected at 10 Hz. The modified APF decision function and prediction module were also checked for compatibility with the 240 Hz control budget. At system level, Monte Carlo benchmarks evaluated velocity sensitivity, latency ablation between proactive and reactive modes, and narrow-passage generalisation. This layered validation made it easier to attribute failures to perception noise, prediction tuning, avoidance logic, or workspace geometry.

5. Core Algorithm Design: Predictive Control and Integrated Safety Logic

5.1 The Necessity of Predictive State Estimation

Purely reactive avoidance is sensitive to sensor noise and sampling delay. To model this constraint, the system does not read obstacle coordinates directly from the physics engine; instead, “3D vision” is implemented as a **synthetic RGB-D (colour-depth) pipeline**. A 10 Hz dual-camera CameraSystem captures depth maps, removes static objects through segmentation masks, and back-projects valid pixels into a 3D point cloud. The dynamic obstacle's point-cloud mean becomes the Kalman filter observation for short-horizon state estimation, while its geometric extent is handled separately through an axis-aligned bounding box (AABB) and clearance checks.

Both virtual cameras are rendered by PyBullet at 10 Hz in the same **fixed PyBullet world frame**, whose origin is the fixed Franka Panda base at (0, 0, 0). In this frame, **+X points from the robot base into the work area, +Y runs laterally from the pickup side towards the tray, and +Z points upward**. Their raw images are not merged; each depth image is segmented and back-projected using its own view and projection matrices. The top-down camera is positioned at (0.8, 0, 0.6) m and looks down towards (0.4, 0, 0) m from above the +X side, giving the planar AABB and point-cloud mean used as the Kalman position observation. The side-view camera is positioned at (0.5, -0.8, 0.4) m and looks across the workspace towards (0.5, 0, 0.3) m from the negative-Y side, so it can estimate obstacle height and vertical clearance. These outputs are fused at the controller level: the AABB supports closest-point avoidance, while the clearance estimate decides whether overhead crossing is safe. No additional extrinsic calibration is needed because both cameras share the PyBullet world frame. Each valid depth pixel is first converted into normalised device

coordinates (x_{ndc}, y_{ndc}) and then back-projected via

$$[X_h, Y_h, Z_h, W_h]^T = (PV)^{-1} [x_{ndc}, y_{ndc}, 2d - 1, 1]^T$$

$$[X, Y, Z]^T = \frac{1}{W_h} [X_h, Y_h, Z_h]^T$$

where d in [0, 1] is the normalised depth value, and P and V are the projection and view matrices of the camera that produced the pixel. The subscript h denotes homogeneous coordinates. After division by the homogeneous coordinate W_h , [X, Y, Z] is a 3D point in the shared PyBullet world frame. Segmentation masks remove the robot, tray, plane, and payload cube before back-projection. The top-down cloud gives the AABB and mean position for the Kalman filter; the side-view cloud gives clearance height (Z) + 0.08 m for APF mode selection. Table 5.1 summarises these fixed perception settings so that the camera geometry can be separated from the later prediction and avoidance parameters.

Table 5.1. Perception pipeline parameters.

Parameter	Top-down	Side-view
Eye position (m)	(0.8, 0, 0.6)	(0.5, -0.8, 0.4)
Target (m)	(0.4, 0, 0)	(0.5, 0, 0.3)
Field of view / resolution	60 deg / 128 x 128	50 deg / 160 x 120
Near / far clip (m)	0.1 / 2.0	0.1 / 2.0
Pixel stride	4	2
Height threshold (m)	0.08	0.05
Output	AABB + point-cloud mean	Z + 0.08 m

5.2 Mathematical Modelling and State-Space Formulation

Although the obstacle exhibits stochastic multi-pattern motion in the simulation environment, the predictor approximates its short-horizon dynamics using a **6D linear constant-velocity state model** with process noise. In this formulation, the obstacle is represented as a point mass for state estimation purposes, since the Kalman Filter is used to track the centroid motion rather than the full obstacle geometry. Based on the standard discrete-time state-space formulation of the Kalman Filter (Welch and Bishop, 1995), a 6D state vector is defined to include both 3D position and 3D velocity components. Here, k indexes the 10 Hz visual update, $p = (p_x, p_y, p_z)$ is the obstacle-centroid position in the shared world frame, and $v = (v_x, v_y, v_z)$ is its estimated velocity:

$$x_k = [p_x, p_y, p_z, v_x, v_y, v_z]^T$$

The state transition matrix F follows a constant-velocity model: position is propagated by current velocity, while velocity is assumed locally constant. Because filtering is driven by 10 Hz observations rather than the 240 Hz physics step, Δt uses the measured interval between visual updates (nominally 0.1 s), and F is rebuilt on each update. In block form, this 6 x 6 matrix maps velocity into position through $\Delta t I_3$, keeps velocity unchanged through the lower-right I_3 block, and uses 0_3 for the lower-left block:

$$F = \begin{bmatrix} I_3 & \Delta t I_3 \\ 0_3 & I_3 \end{bmatrix}$$

Here, I_3 denotes the 3 x 3 identity matrix and 0_3 denotes the 3 x 3 zero matrix. This linear formulation was chosen because it is computationally lightweight and suitable for short-horizon prediction within the 240 Hz control loop. Rather than modelling the full complexity of obstacle motion, it provides a fast and stable approximation that supports real-time avoidance under sparse visual updates.

5.3 Algorithmic Implementation of the Predict-Correct Cycle

The prediction module implements a recursive Kalman filter cycle. As illustrated in Figure 5.3, the algorithm consists of two stages: state prediction and measurement correction. This design allows the robot to maintain a stable trajectory estimate between camera updates by relying on the predicted motion.

```

def update(self, observed_position, timestamp=None):
    if observed_position is None:
        return

    obs = np.array(observed_position)
    self.position_history.append(obs.copy())

    # Initialization: Use the first observation as the starting state
    if not self.initialized:
        self.state = np.concatenate([obs, [0, 0, 0]]) # Pos + zero velocity
        self.covariance = np.eye(6) * 0.1
        self.initialized = True
        return

    # Phase 1: Physical Prediction (Time Update)
    # state_pred = F @ current_state
    state_pred = self.F @ self.state
    # cov_pred = F @ P @ F.T + Q
    cov_pred = self.F @ self.covariance @ self.F.T + self.Q

    # Phase 2: Measurement Correction (Update)
    y = obs - self.H @ state_pred # Innovation
    S = self.H @ cov_pred @ self.H.T + self.R # Innovation Covariance
    K = cov_pred @ self.H.T @ np.linalg.inv(S) # Optimal Kalman Gain

    # Refine state and covariance estimates
    self.state = state_pred + K @ y
    self.covariance = (np.eye(6) - K @ self.H) @ cov_pred

    # State Extraction for Avoidance Logic
    self.velocity_estimate = self.state[3:6]
    self.speed_magnitude = np.linalg.norm(self.velocity_estimate)
    self.is_moving = self.speed_magnitude > 0.0005 # Motion detection threshold

```

Figure 5.3: Core 6D Kalman Filter Update Logic

5.4 Engineering Trade-offs: Tuning Q and R Matrices

Kalman filter performance depends on the process-noise covariance Q and measurement-noise covariance R . These matrices were not obtained from formal camera calibration; they were set as diagonal matrices and **tuned empirically** under the same 10 Hz visual update rate as the controller. The final implementation used $Q = \text{diag}(0.001, 0.001, 0.001, 0.01, 0.01, 0.01)$ for the 6D state $[p_x, p_y, p_z, v_x, v_y, v_z]$, and $R = \text{diag}(0.01, 0.01, 0.01)$ for the 3D position measurement. The setting balances responsiveness and smoothness and should be interpreted as engineering calibration for this simulated benchmark, not as a physical camera noise model.

5.5 Motion-Trend-Aware Prediction Horizon and Confidence Blending

The predictor does not perform semantic intent recognition. It uses a **lightweight**

motion-trend heuristic derived from obstacle velocity in the same PyBullet world frame defined in Section 5.1: +Y follows the pickup-to-tray corridor, +Z points upward, and +X spans the lateral approach direction. Because the scripted dynamic obstacle mainly enters and exits the interaction zone along X, the sign of v_x is used as the trend cue: negative v_x means approaching the robot-workspace corridor, positive v_x means leaving it, and near-zero v_x means approximately stationary. This trend adapts the prediction horizon through a baseline lead-time factor of 1.5 and a multiplier: 1.5 when approaching, 0.5 when leaving, and 1.0 otherwise. This is a scene-specific benchmark heuristic, not a general intent-recognition model.

To reduce abrupt switching, the predictor also computes a **heuristic confidence score** for each forward prediction. This score is based on recent observation coverage, temporal decay over the prediction horizon, and the stability of recent motion. The confidence value is then converted into a blend weight α , which determines how strongly the avoidance module should rely on the predicted future position rather than the current filtered state. In other words, the system does not treat every prediction as equally trustworthy; it increases the influence of prediction only when recent evidence is sufficiently stable.

This mechanism is therefore better understood as motion-trend-aware prediction and confidence blending than as semantic intent recognition. Its purpose is to adjust the controller's short-horizon response according to both current motion tendency and the estimated reliability of the recent prediction.

5.6 Integrated Safety Logic: Hierarchical Avoidance and Payload-Aware Boundary Adaptation

Hierarchical control is common in HRC because it separates high-level cooperation logic from low-level motion and safety modules (Darvish et al., 2021). Here, the predictor's state estimate feeds a **hierarchical safety logic** rather than a single reactive controller. First, the predicted obstacle state sets a graded threat response, ranging from cautious proceed to pre-emptive avoidance and emergency avoidance. Second, the controller selects the local manoeuvre using workspace geometry and obstacle clearance instead of applying one uniform repulsive response.

The avoidance module first checks whether enough vertical clearance exists above the obstacle. If so, it uses overhead crossing, keeping the end effector and any payload above the estimated clearance height while moving towards the target. If clearance is limited, it switches to alternatives such as lifting, far-transport motion, or lateral escape. The behaviour is therefore a structured combination of clearance-aware mode selection and local APF motion generation: prediction determines

urgency, while rule-based avoidance decides how to respond in the current workspace.

Concretely, the predicted state enters the APF along two channels. First, an effective obstacle centre p_{eff} is computed from the filtered centroid p_{now} and its forward prediction p_{pred} :

$$p_{eff} = (1 - c) p_{now} + c p_{pred}$$

where c in $[0, 1]$ is the heuristic prediction confidence from Section 5.5, and p_{now} and p_{pred} are both obstacle-centroid positions in the shared world frame. A low c keeps the APF close to the current filtered observation, while a high c shifts the reference towards the forward prediction. This effective centre translates the reconstructed AABB forward in time, and the APF reference is taken from the closest point on the shifted obstacle volume rather than from the centroid alone. If the obstacle is approaching, the reference is biased by 5 cm along the obstacle-to-robot axis to trigger avoidance slightly earlier. The predictor also adjusts the warning radius d_2 (Table 5.6), expanding it by up to 15 cm for fast approaching motion or multiplying it by 0.85 when the obstacle is leaving. Thus prediction affects both where the obstacle is treated as being and how strongly the APF responds.

Table 5.6 lists the threshold values used by these avoidance rules. Here, d_1 and d_2 are clearance thresholds used by the APF safety logic, the threat score is a normalised heuristic estimate of approaching-obstacle risk, and c_{max} limits how much the controller can rely on the predicted obstacle position.

Table 5.6. Key APF parameters driven by predicted obstacle motion.

Parameter	Value	Meaning
d_1 : hard-stop distance threshold	0.15 m	Minimum allowed clearance before hard-stop safety logic is triggered.
d_2 : baseline APF warning threshold	0.40 m	Distance at which APF repulsion begins under normal or cautious motion.
d_2 : preemptive warning threshold	0.48 m	Expanded warning distance when the normalised threat score is above 0.4.
d_2 : emergency warning threshold	0.55 m	Expanded warning distance when the normalised threat score is above 0.7.
c_{max} : prediction-weight cap	0.9	Maximum confidence weight used when blending current and predicted obstacle positions.
Approaching reference bias	0.05 m	Forward shift applied to the effective

		obstacle reference when the obstacle is approaching.
d_2 : speed-based expansion limit	up to 0.15 m	Maximum additional warning-distance increase for fast approaching motion.
d_2 : leaving-obstacle relaxation factor	0.85	Reduces the warning threshold when the obstacle is moving away.
Waypoint step size (high risk / otherwise)	0.03 / 0.05 m	Smaller Cartesian step under higher risk; larger step under lower risk.

The safety boundary is further adapted using motion and payload information. Within the avoidance module, the effective safe distance expands for approaching obstacles and relaxes slightly for leaving obstacles. After grasping, payload-dependent horizontal and bottom offsets extend the protected geometry. Clearance is measured from the closest point on the reconstructed obstacle AABB, so the controller evaluates edge-to-edge space between the **combined robot-payload body** and the occupied obstacle volume rather than protecting the arm alone.

6. System Evaluation and Results Analysis

This chapter evaluates the proposed framework against the core objectives established in the preceding chapters. Table 6.1 links each evaluation component to a specific objective, the evidence used to assess it, the main outcome, and the scope within which the resulting claim should be interpreted. The primary velocity-sensitivity benchmark design and the complementary quantitative analyses are then presented in the following sections.

Table 6.1: Evaluation roadmap linking objectives, evidence, key findings, and scope of claims.

Objective	Test focus	Evidence	Key result	Scope of claim
O1	End-to-end safety and task performance under dynamic human-like interference.	Sections 6.1–6.2.2; 150 Monte Carlo trials; five speed levels; success criteria, minimum distance, collision frames, and execution time.	Success rate declines with obstacle speed: Slow 93.3%, Medium 93.3%, Fast 86.7%, Extreme 83.3%, Insane 70.0% (benchmark mean 85.3%). Mean minimum clearance for successful zero-collision trials stays within 0.150–	Defines the operating envelope of the current controller under the strictest zero-collision criterion. Reveals that residual failures become increasingly contact-related at higher speeds, while low-speed failures are rare timeout events. The

			0.166 m, with the Insane stress-test condition producing the lowest mean value at approximately 0.150 m.	result defines an operating envelope rather than a collision-free guarantee under extreme disturbance.
O2	Robustness of the perception-prediction layer under noisy observations.	Section 6.2.3; Gaussian noise with $\sigma=2$ cm added to obstacle observations; RMSE comparison before and after filtering.	RMSE decreases from about 3.4 cm to about 1.5 cm, a roughly 56% reduction.	Supports robustness at the tracking layer only. It does not by itself prove full end-to-end task success under all noise conditions.
O3	Generalisation across related workspace and obstacle configurations.	Section 6.2.4; narrow-passage benchmark; gap widths from 0.20 m to 0.40 m; 30 trials per width, 150 trials in total; same success criteria as the main benchmark.	96.7% overall task success across 150 narrow-passage trials, with zero collision frames in all trials. Execution time remains stable across gap widths, and the mean minimum clearance remains positive even in the narrowest 0.20 m passage.	Supports configuration-level generalisation in related simulated settings. It should not be interpreted as evidence of cross-robot or real-world generalisation.
O4	Added value of predictive estimation for maintaining safety margin under perception latency.	Section 6.2.6; ablation comparing APF+KF against APF-only under 10 Hz visual latency across five obstacle-speed regimes (0.001, 0.003, 0.005, 0.007, 0.017 m/step); 30 trials per condition.	The ablation reveals a speed-dependent cross-over: APF+KF improves clearance at low-to-moderate speeds, while the reactive baseline achieves larger clearance at the highest speed settings.	Evidence for a bounded prediction benefit under delayed feedback; supports using Kalman prediction as an auxiliary response layer rather than as the sole source of safety.

6.1 Velocity-Sensitivity Benchmark Design

Because static datasets cannot capture continuous HRC interaction, the APF+KF framework was evaluated in PyBullet using a **150-trial velocity-sensitivity benchmark**. This primary benchmark exposed the robot to repeated heterogeneous dynamic disturbances rather than a single fixed obstacle path.

Each trial reset the simulation, Franka Panda, tray, and payload cube to the same initial configuration. A dynamic obstacle was then assigned a base speed and bounded workspace, and the robot executed one complete pick-and-place task under the proposed controller. The system recorded minimum obstacle distance, collision frames, execution time, and final placement. A run succeeded only if all kinematic, safety, and timing criteria were met, making obstacle speed the main manipulated variable.

6.1.1 Simulated Data Composition and Interference Setup

The dynamic obstacle was a 0.8 m × 0.12 m × 0.12 m box approximating a worker-arm volume. Its motion was bounded from 0.35 to 0.65 m in X, -0.15 to 0.15 m in Y, and 0.15 to 0.35 m in Z. The narrower Y range kept the obstacle inside the main camera's field of view, reducing partial-occlusion effects on AABB measurement. The zone overlapped the robot's pick-and-place corridor between pickup and tray, so interference occurred mainly during payload transport rather than grasping or final placement.

This velocity-sensitivity benchmark used four stochastic motion families rather than one deterministic path: Linear approach (30%), Zigzag (25%), Circular (20%), and Random Walk (25%). This exposed the Kalman filter and APF logic to both smooth and abruptly changing obstacle motion. Figure 6.1 shows a representative scene. The green box is the top-view reconstructed AABB, representing the estimated occupied obstacle volume; the point-cloud mean is passed to the Kalman filter as the motion-prediction observation. The green box is therefore a perception-level estimate, not the full safety boundary used by the controller.

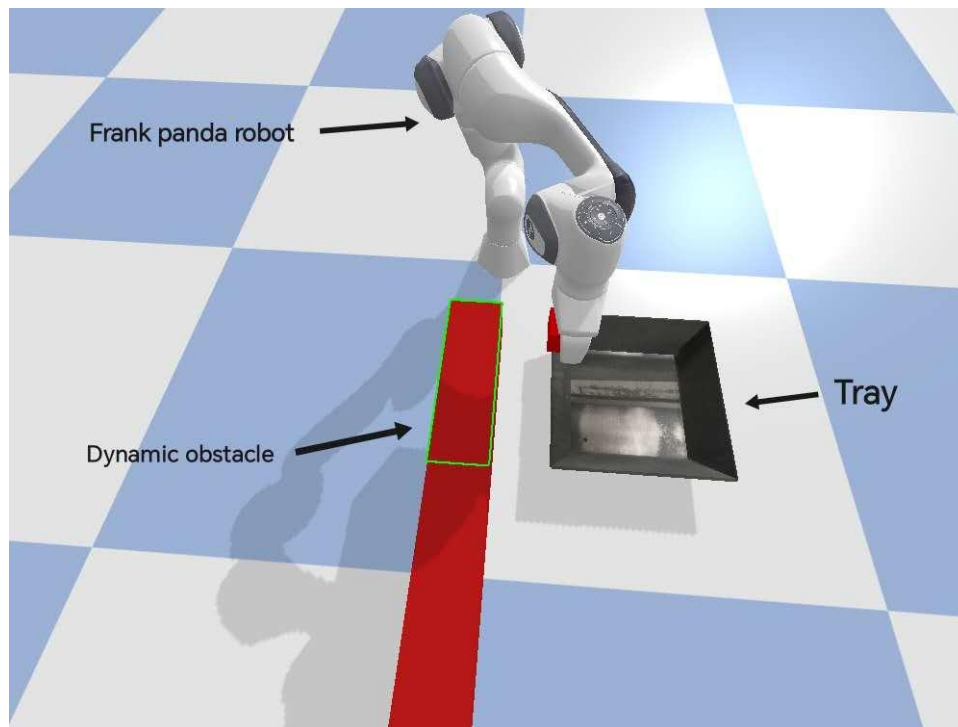


Figure 6.1: Experimental setup of the velocity-sensitivity Monte Carlo benchmark in PyBullet.

6.1.2 Independent Variable: Dynamic Interference

The independent variable was obstacle base speed. Five levels were tested: Slow (0.001 m per simulation step), Medium (0.003), Fast (0.005), Extreme (0.007),

and an “Insane” stress-test setting (0.017). These values were selected during pilot testing to create increasingly difficult disturbance levels, not to model realistic worker motion. They therefore represent controlled benchmark stress levels, allowing analysis of how predictive avoidance behaves as disturbance speed increases.

6.1.3 Dependent Variables and Multi-Dimensional Success Criteria

Unlike conventional benchmarks that focus only on task completion, this study evaluated performance using strict constraints across kinematic, safety, and temporal dimensions. A trial was classified as a failure if any one of the following conditions was violated.

- (i) Kinematic Deviation: the payload was dropped, missed the tray, or its final placement lay outside the target tolerance. Operationally, the final XY distance from the tray centre had to remain below 0.15 m and the final payload height had to remain above 0.015 m. Otherwise, the run was treated as a placement failure.
- (ii) Safety Violation: a trial was classified as a safety failure if any collision frame was recorded between the manipulator or the payload and the dynamic obstacle during the trial. This strict zero-collision criterion treats even a single frame of contact as unsafe, making the safety metric conservative and easy to reproduce. The total collision count and the maximum consecutive contact duration were still logged per trial as continuous diagnostics, and are used in later sections to discuss the sim-to-real implications of sustained contact events.
- (iii) Temporal Deadlock: the execution time exceeded 60.0 s, suggesting that the Artificial Potential Field (APF) had trapped the robot in a local minimum or otherwise prevented timely task completion. The 60 s budget was chosen to provide a comfortable margin over the 20–40 s nominal pick-and-place duration observed in unobstructed trials, while still allowing genuinely deadlocked trajectories to be identified.

Alongside binary task outcome, three continuous metrics were recorded. Minimum safe distance was the closest edge-to-edge clearance between the protected robot-payload geometry and reconstructed obstacle AABB, not a centroid-to-centroid distance. The reported standard deviation describes how this minimum clearance varied across repeated trials within the same speed or gap condition. Collision frames were counted whenever PyBullet reported contact between the manipulator or payload and the dynamic obstacle; maximum consecutive contact duration was retained as a sustained-contact diagnostic. Unless stated otherwise, clearance summaries use successful zero-collision trials so stability plots describe safe task executions rather than failed collision runs.

6.1.4 Sample Size

To reduce randomness caused by trajectory-specific motion patterns such as zigzag motion and random walks, 30 independent trials were conducted for each of the five speed levels, giving a total of 150 simulation runs. This sample size provided a practical balance between computational cost and statistical stability, allowing each speed setting to be evaluated with repeated exposure to heterogeneous obstacle motion rather than a single trial outcome.

6.2 Quantitative Results Analysis

6.2.1 Task Completion Rate Across Speed Regimes

Under the strict zero-collision criterion in Section 6.1.3, success declined with obstacle speed, from 93.3% at Slow and Medium to 70.0% at Insane, with a benchmark mean of 85.3% (Figure 6.2.1). Across the 150 velocity-sensitivity trials, 22 runs failed. Contact-related failures concentrated at higher speeds, while the two Slow-regime failures were timeouts, showing that timely avoidance becomes harder as disturbance speed increases.

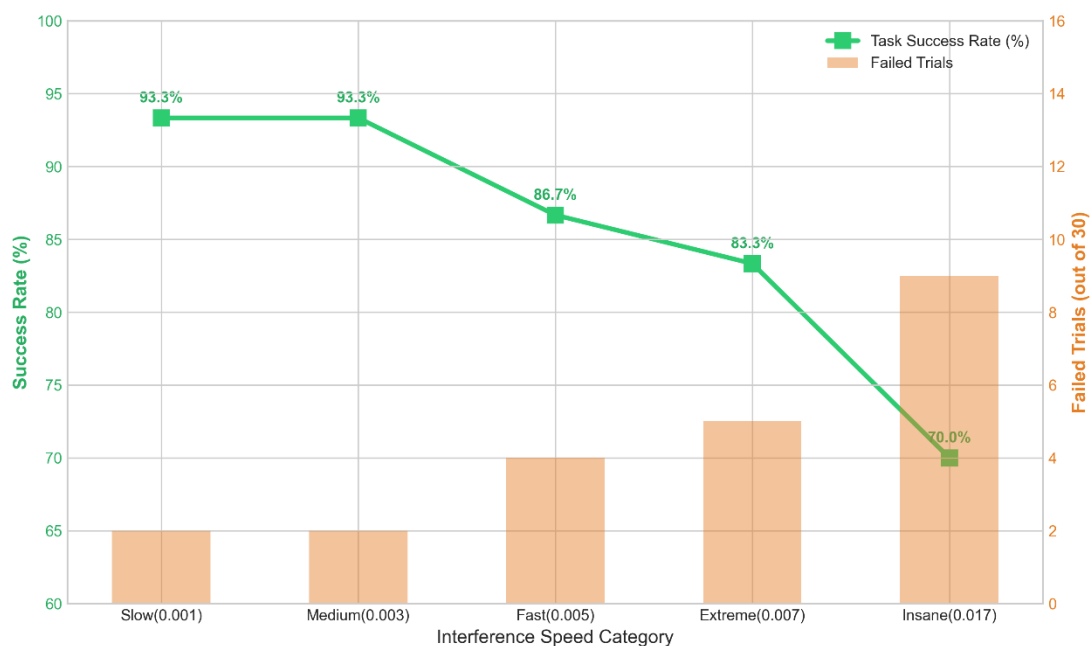


Figure 6.2.1: Success Rate vs. Obstacle Speed and Associated Collision-Failure Analysis

Analysis of the failure logs helps explain these results:

At the Fast, Extreme, and Insane speeds (0.005–0.017 m per simulation step), failures are dominated by contact-related events: 18 failed runs occurred across these three regimes, and all involved recorded contact frames. The longest sustained contact episode reached 91 consecutive frames, approximately 379 ms at 240 Hz.

Under these conditions the obstacle traverses a substantial distance between successive 10 Hz visual updates; although the Kalman filter runs at 240 Hz, the robot's physical inertia limits how quickly the joint-space avoidance manoeuvre can take effect, which allows measurable contact episodes to occur.

At the Slow and Medium speeds (0.001–0.003), failures remained sparse. Four trials out of 60 failed: two Slow-regime runs exceeded the 60 s time budget without collision, while the Medium-regime failures consisted of one contact-only collision failure and one drop/miss failure that also recorded contact frames. This indicates that low-speed failures are rare and mainly associated with timing or isolated placement-contact events, whereas high-speed failures are primarily safety-contact limited.

6.2.2 Statistical Stability and Consistency of the Digital Safety Envelope

Figure 6.2.2 shows that the controller maintained a **positive edge-to-edge clearance envelope** across the tested speed regimes. These distances are tighter than earlier centroid-based measurements because clearance is now computed from the AABB surface and payload-aware boundary, giving a more physically meaningful safety margin.

To complement the primary benchmark, the following two sections further examine robustness to noisy perception and generalisation across related workspace and obstacle configurations.

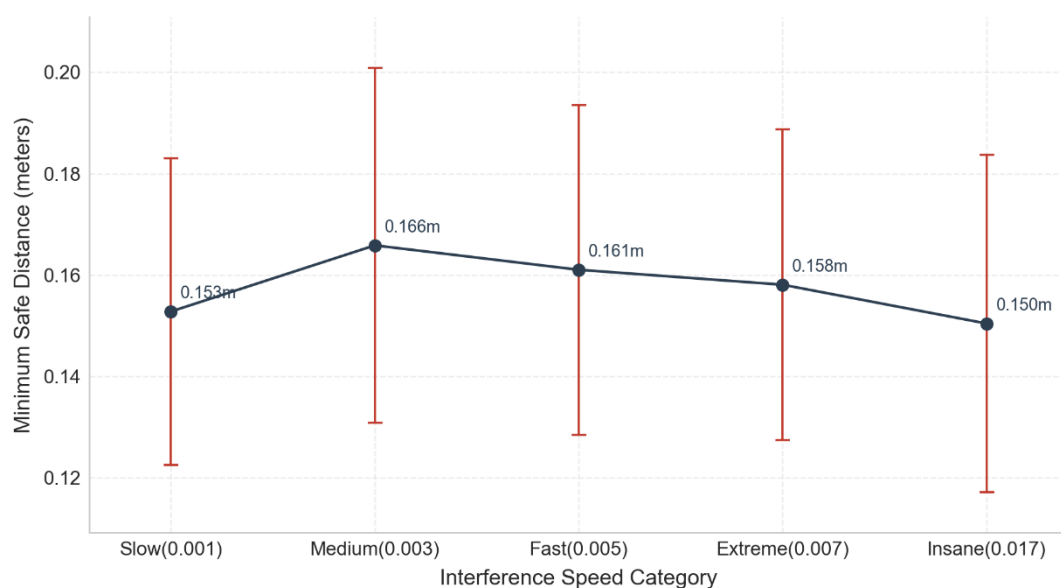


Figure 6.2.2: Mean Minimum Distance and $\pm 1\sigma$ Envelope Across Obstacle Speeds

6.2.3 Perception Robustness Under Noisy Observations

Robustness was first evaluated at the perception-tracking layer by adding 2 cm Gaussian noise to obstacle position observations. As Figure 6.2.3 shows, Kalman filtering reduced root mean square error (RMSE) from about 3.4 cm to about 1.5 cm, a roughly 56% error reduction. This indicates that filtering improves the obstacle-state estimate used downstream, but it does not by itself prove better end-to-end task success or avoidance quality. Its system-level value is tested separately in Section 6.2.6, which examines clearance, contact exposure, and jerk under latency-stressed conditions.

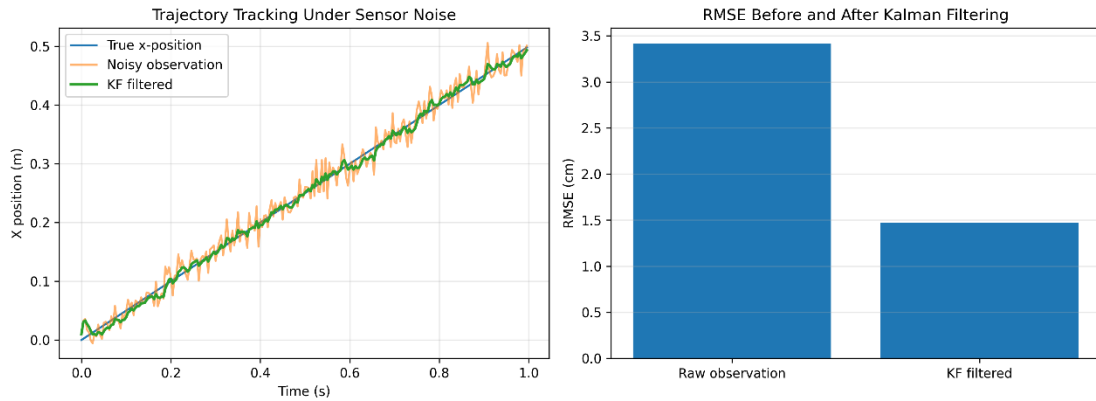


Figure 6.2.3: Effect of Kalman filtering on noisy obstacle observations. The left panel shows that the filtered trajectory follows the ground-truth motion more closely than the raw noisy observation; the right panel compares RMSE before and after filtering.

6.2.4 Generalisation Across Workspace and Obstacle Configurations

To test generalisation beyond one calibrated scene, a narrow-passage benchmark varied the gap between two static obstacles from 0.20 m to 0.40 m with all controller parameters fixed. For each gap, 30 trials were run with small cube-position perturbations, giving 150 trials. Using the Section 6.1.3 criteria, APF+KF achieved 96.7% overall success. All 150 trials were collision-free; the five failures were placement misses, not safety violations. Success rates were 96.7%, 96.7%, 100.0%, 96.7%, and 93.3% from 0.40 m to 0.20 m. Execution time remained stable (34.51 ± 1.45 s to 35.21 ± 0.59 s), and mean clearance stayed positive, with 2.94 ± 0.63 cm in the narrowest passage.

Figure 6.2.4: Generalisation Across Narrow-Passage Workspace Configurations

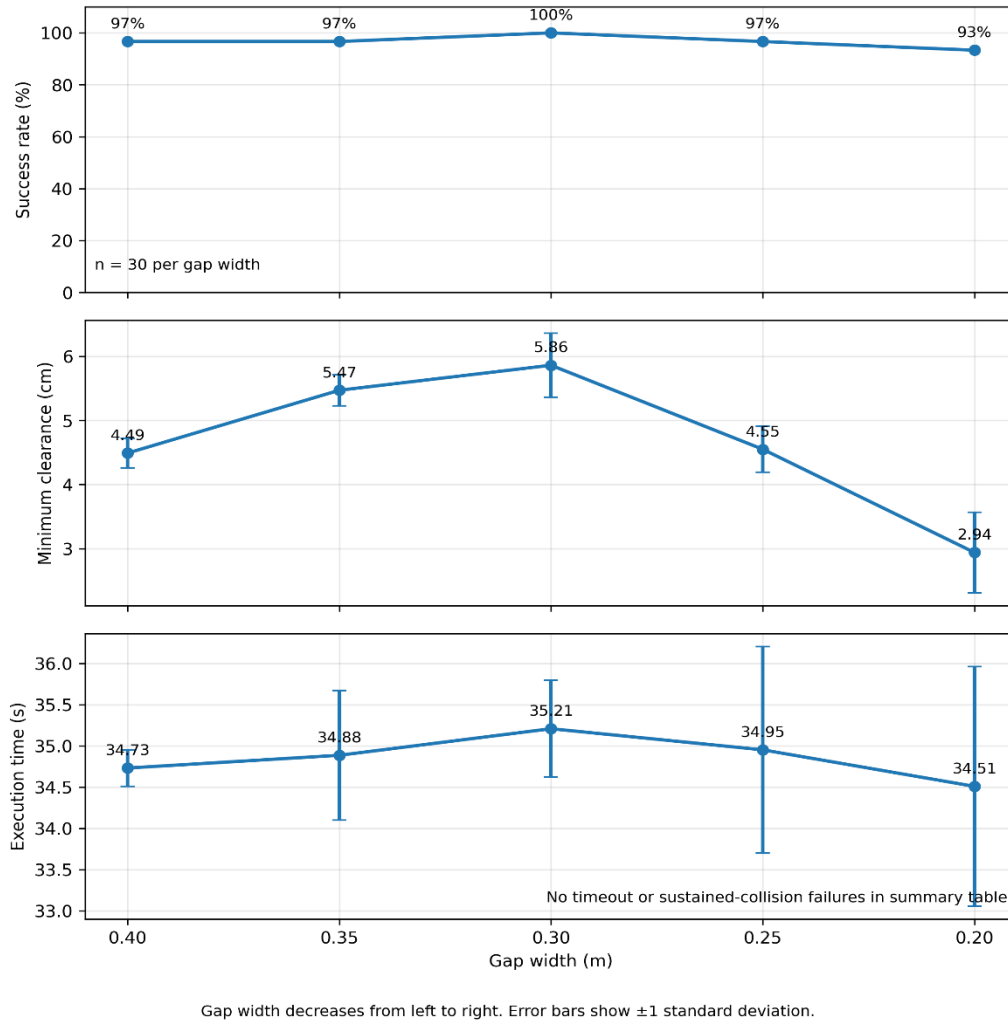


Figure 6.2.4: Benchmark results across narrow-passage workspace configurations

6.2.5 Fault Tolerance and the Sim-to-Real Gap Challenge

Collision logs show that outside the optimal velocity range, contact can persist beyond momentary overlap. In high-speed contact-involved failures, episodes reached 91 consecutive frames (379 ms at 240 Hz). PyBullet treats these as digital overlaps, but in a real factory they would likely trigger emergency stops. Consistent with Peng et al. (2018), unmodelled dynamics and contact friction can make high-speed control behaviour validated only in simulation less reliable on hardware. Physical deployment would therefore require additional safety control, especially compliant joint-level impedance control.

6.2.6 Ablation Study: Effect of Prediction on Clearance Margin under Delayed Perception

To isolate predictive state estimation under delayed perception, the full APF+KF

controller was compared with a reactive APF-only baseline. The reactive mode used the delayed obstacle AABB centre directly and disabled KF forward prediction, motion-trend-aware warning-radius modulation, and pre-emptive/emergency expansion of d_2 .

A 10 Hz delayed-observation setting represented sparse visual feedback. Rather than using binary task success, the ablation focused on mean minimum safety distance, mean contact frames, and mean jerk, matching the predictor's role as a safety-margin and response-modulation layer rather than a complete planner replacement.

Figure 6.2.6 shows a **speed-dependent cross-over**. APF+KF maintained larger clearance at low-to-moderate speeds: +6.6 mm at 0.001 m/step, +3.1 mm at 0.003, and +10.9 mm at 0.005. At higher speeds this advantage disappeared: the reactive baseline was higher by +2.2 mm at 0.007 and +17.6 mm at 0.017. Mixed contact-frame and jerk results show that prediction improves separation within part of the operating envelope but is **not uniformly better across all dynamic regimes**.

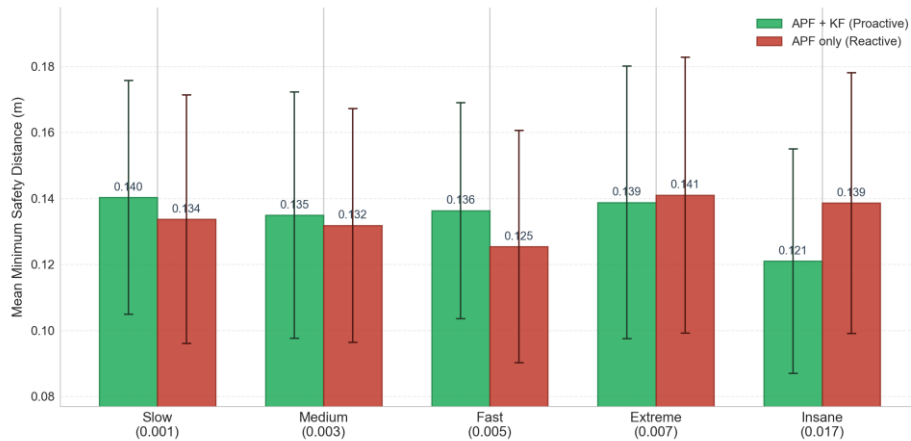


Figure 6.2.6: Mean minimum safety distance comparison between APF+KF and reactive APF-only controllers

6.3 Critical Evaluation and Limitations

6.3.1 Local Minima and the Planning Landscape

Within APF+KF, **APF remains a local planner**. Although it performs well in these experiments, dense multiple-obstacle fields can create overlapping repulsive fields and local minima. Modified APF methods address this issue in the literature (Ge and Cui, 2000), while sampling-based planners such as RRT-style planners and mobile-robot dynamic local planners such as DWA illustrate alternative strategies for some spatial traps (Fox et al., 1997; Karaman and Frazzoli, 2011). They were not used here because sampling and optimisation overhead would threaten the 4.16 ms timing

budget of the 240 Hz predictive HRC loop.

6.3.2 Kinematic Singularities and Joint Constraints

The current algorithm computes repulsive motion in Cartesian task space and does not explicitly penalise Jacobian condition number or joint-limit proximity. During extreme avoidance, the manipulator may therefore approach singularities or poor joint configurations even when clearance is maintained. This reflects the framework's priority of deterministic low-latency avoidance over higher-dimensional optimisation. Future work could add null-space projection or joint-limit-aware secondary objectives to improve kinematic robustness.

7. Discussion and Reflection

7.1 Project Summary and Core Contributions

This project demonstrated that a lightweight real-time safety architecture can support dynamic Human-Robot Collaboration under delayed visual feedback by integrating perception, short-horizon prediction, hierarchical APF avoidance, and payload-aware safety checking within a 240 Hz control loop. Its main contribution is not a globally optimal planner, but a **practical geometric safety framework** in which AABB-aware clearance estimation and robot-payload boundary adaptation provide the core safety structure, while Kalman prediction acts as an auxiliary layer for earlier response under suitable motion conditions.

7.2 Critical Evaluation of Algorithmic Boundaries Across Benchmarks

Across the benchmarks, the framework performs most reliably under low-to-moderate dynamic interference and in narrow-passage layouts where the obstacles are static. The velocity-sensitivity benchmark shows the speed boundary most clearly: the two Slow-regime failures were timeout events rather than collisions, while higher-speed failures were dominated by contact-related events. The narrow-passage benchmark adds a separate boundary condition: all 150 trials were collision-free, but small placement misses appeared as the gap narrowed. The ablation study further shows that Kalman prediction has a **bounded operating envelope**: it improves clearance when the predicted lead-time displacement remains compatible with the workspace scale, but can lose advantage when linear extrapolation at very high obstacle speed becomes too aggressive for the local controller to exploit safely.

7.3 Robustness and the Sim-to-Real Gap

Although the framework maintained a relatively stable digital safety envelope across the tested speed range, this should not be interpreted as evidence that the

same level of safety would hold in physical deployment. At extreme speeds, contacts that are digitally resolved in PyBullet could correspond to protective stops, hardware faults, or unsafe contact events on a real manipulator. The more important conclusion is therefore that prediction should be treated as a **risk-reduction layer rather than as a complete safety guarantee**, and that compliant control and hardware-grounded validation remain necessary before deployment.

7.4 Ethical, Social, and Professional Responsibilities in HRC

In HRC, the ethical priority is not only task efficiency but reducing harm when humans and robots share a workspace. This project treats safety as a control requirement, yet the results also show that **predictive avoidance is not a safety guarantee**. Performance degrades at extreme obstacle speeds because the manipulator cannot always respond quickly enough to predicted motion. Prediction should therefore be presented as risk reduction, not proof that dangerous contact has been eliminated.

A second responsibility concerns the gap between simulation and real deployment. In PyBullet, contact events can be counted and analysed without causing physical damage. In a real collaborative workspace, the same interaction could trigger safety stops, damage hardware, or place a human operator at risk. For that reason, the current framework should not be presented as ready for deployment in a physical cyber-factory environment. The project therefore takes a conservative position: the present results demonstrate the value of short-horizon prediction in simulation, but they do not remove the need for **further validation, compliant control, and hardware-level safeguards** before any real-world use.

There is also a professional obligation to design for failure, not only for nominal success. In edge-case scenarios, obstacle motion may become highly erratic, sensor updates may be sparse, and prediction confidence may drop. Under those conditions, the correct engineering response is not to preserve task completion at all costs, but to fall back to safer behaviour, such as maintaining larger separation distances or triggering a protective stop. This reflects a broader principle of responsible autonomous-systems engineering: when uncertainty increases, the system should become more conservative rather than more assertive.

The social implications of HRC should also be considered realistically. Safety-oriented automation can reduce worker exposure to repetitive or hazardous operations, but its value should not be framed only in terms of labour replacement or productivity gain. In practice, systems of this kind are more defensible when they support safer human supervision, maintenance, and coordination roles rather than assuming that human participation can simply be removed. The social benefit

therefore depends not only on whether the robot performs efficiently, but on whether the technology is introduced in a way that genuinely improves workplace safety and preserves meaningful human oversight.

Professional conduct also requires restraint in how intelligent-system claims are reported. Under the BCS Code of Conduct (BCS, no date), such claims require transparent testing, clear limitations, and cautious interpretation. This project therefore reports failure modes such as temporal deadlock, kinematic latency, and the sim-to-real gap, because safety-critical systems must be judged by how they fail as well as how they succeed.

7.5 Personal Performance Evaluation and Professional Growth Reflection

A key project lesson is that systems engineering depends on **process discipline** as much as technical implementation. The staged strategy helped maintain progress, but my time planning underestimated the effort required to tune the Kalman filter's Q and R matrices. Future projects should include **larger contingency buffers** for calibration, debugging, and repeated validation rather than assuming that mathematically sound methods integrate smoothly.

A second area of growth was learning how to make technical decisions under operational constraints rather than on theoretical appeal alone. This project made me more disciplined in selecting methods that could be justified not only academically, but also in terms of timing, integration cost, and full-system compatibility. The main lesson was that a method is valuable only when it remains workable within the real limits of the system in which it must operate.

The project also changed the way I think about verification. Early in development, I tended to judge progress by whether the robot could complete an individual task successfully. As the project matured, I recognised that this was not an adequate basis for evaluating safety-oriented behaviour. This led first to a 150-trial velocity-sensitivity Monte Carlo benchmark and then to a separate 150-trial narrow-passage benchmark, shifting testing from single-run demonstration towards systematic stress evaluation across multiple operating conditions. That transition was significant for my own professional development, because it moved my approach from script-level implementation towards a more rigorous systems-engineering mindset centred on repeatability, robustness, and evidence-based assessment.

Overall, this project strengthened my ability to balance theoretical ambition with practical constraints, and to treat debugging, benchmarking, and design trade-offs as core engineering activities rather than secondary tasks. The most important professional insight I gained is that reliable intelligent systems are not produced by maximising model complexity, but by making justified design choices, validating them

systematically, and being explicit about the limits within which the resulting claims remain credible.

8. Conclusion and Future Work

This project demonstrates a real-time predictive safety framework for Human-Robot Collaboration. At the same time, the limitations observed under extreme velocity conditions highlight several areas for future improvement.

8.1 More Robust Short-Horizon Prediction and Deadlock Recovery

Future work should focus on the two main limitations revealed by the velocity-sensitivity benchmark and ablation study: **high-speed degradation of the linear short-horizon predictor**, and timeout behaviour under prolonged local interference. A practical next step would be to evaluate an **adaptive prediction horizon** that contracts as observed obstacle speed increases, together with a lightweight motion-mode classifier that can distinguish stable approach motion from abrupt or oscillatory trajectories. This could preserve the benefit of early response at low-to-moderate speeds while reducing over-extrapolation at the highest disturbance levels.

8.2 Bridging the Sim-to-Real Gap Through Compliance and Hardware Validation

The next stage should validate the framework under **hardware-in-the-loop or staged real-robot testing**. This would expose real sensing latency, actuator saturation, friction, compliance, and contact behaviour, and allow comparison with collaborative-robotics safety expectations in ISO/TS 15066:2016 and the revised ISO 10218-1:2025 (International Organization for Standardization, 2016, 2025). Compliant Cartesian or joint-level impedance control would be a natural extension for safer response during rapid contact or near-contact events.

8.3 More Reliable Multi-Rate Perception and Confidence-Aware Fallback Behaviour

A more immediate extension of the current project would be to strengthen the perception-prediction interface rather than moving directly to semantic intent modelling. At present, the framework already operates under a multi-rate architecture in which visual observations arrive more slowly than the control loop. The latency ablation results show that prediction can help within part of this setting, but that its benefit depends on obstacle speed, prediction horizon, and observation timing. Future work should therefore focus on improving robustness to sparse updates, temporary occlusion, and uncertain obstacle measurements.

A useful extension would make fallback behaviour depend explicitly on prediction

confidence. When confidence drops, the controller could increase separation distance, slow transport, or switch to a more conservative protective mode. The safety response would then reflect both estimated obstacle motion and the reliability of that estimate.

This direction is more closely aligned with the present framework than a move towards sequence-based trajectory prediction methods such as Social Long Short-Term Memory (Social LSTM) (Alahi et al., 2016). The current project has demonstrated the value of short-horizon geometric prediction under strict timing constraints, so the next logical step is to make that prediction pipeline more reliable and more conservative under uncertainty before pursuing higher-level behavioural modelling.

References

- Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Fei-Fei, L. and Savarese, S. (2016) 'Social LSTM: Human trajectory prediction in crowded spaces', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 961-971. doi: 10.1109/CVPR.2016.110.
- Arulampalam, M.S., Maskell, S., Gordon, N. and Clapp, T. (2002) 'A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking', IEEE Transactions on Signal Processing, 50(2), pp. 174-188. doi: 10.1109/78.978374.
- BCS, The Chartered Institute for IT (no date) Code of Conduct. [Online] Available at: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/> [Accessed 9 March 2026].
- Coumans, E. and Bai, Y. (2016-2021) PyBullet: a Python module for physics simulation for robotics, games and machine learning [Computer software]. Available at: <http://pybullet.org> [Accessed 9 March 2026].
- Darvish, K., Simetti, E., Mastrogiovanni, F. and Casalino, G. (2021) 'A hierarchical architecture for human-robot cooperation processes', IEEE Transactions on Robotics, 37(2), pp. 567-586. doi: 10.1109/TRO.2020.3033715.
- Flacco, F., Kroger, T., De Luca, A. and Khatib, O. (2012) 'A depth space approach to human-robot collision avoidance', 2012 IEEE International Conference on Robotics and Automation (ICRA). IEEE, pp. 338-345. doi: 10.1109/ICRA.2012.6225245.
- Fox, D., Burgard, W. and Thrun, S. (1997) 'The dynamic window approach to collision avoidance', IEEE Robotics & Automation Magazine, 4(1), pp. 23-33. doi: 10.1109/100.580977.
- Ge, S.S. and Cui, Y.J. (2000) 'New potential functions for mobile robot path planning', IEEE Transactions on Robotics and Automation, 16(5), pp. 615-620. doi:

10.1109/70.880813.

International Organization for Standardization (2016) ISO/TS 15066:2016 Robots and robotic devices: Collaborative robots. Geneva: ISO.

International Organization for Standardization (2025) ISO 10218-1:2025 Robotics: Safety requirements. Part 1: Industrial robots. Geneva: ISO.

Karaman, S. and Frazzoli, E. (2011) 'Sampling-based algorithms for optimal motion planning', *The International Journal of Robotics Research*, 30(7), pp. 846-894. doi: 10.1177/0278364911406761.

Khatib, O. (1986) 'Real-time obstacle avoidance for manipulators and mobile robots', *The International Journal of Robotics Research*, 5(1), pp. 90-98. doi: 10.1177/027836498600500106.

Koren, Y. and Borenstein, J. (1991) 'Potential field methods and their inherent limitations for mobile robot navigation', *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, pp. 1398-1404. doi: 10.1109/ROBOT.1991.131810.

Mayne, D.Q., Rawlings, J.B., Rao, C.V. and Scokaert, P.O.M. (2000) 'Constrained model predictive control: Stability and optimality', *Automatica*, 36(6), pp. 789-814. doi: 10.1016/S0005-1098(99)00214-9.

Peng, X.B., Andrychowicz, M., Zaremba, W. and Abbeel, P. (2018) 'Sim-to-real transfer of robotic control with dynamics randomization', *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 3803-3810. doi: 10.1109/ICRA.2018.8460528.

Shang, J., Liu, G., Zhang, T., He, H., Tian, G., Li, W. and Liu, Z. (2025) 'Adaptive obstacle avoidance using vision-based dynamic prediction and strategic motion planning', *IET Cyber-Systems and Robotics*, 7(1), e70034. doi: 10.1049/csy2.70034.

Tonola, C., Faroni, M., Abdolshah, S., Hamad, M., Haddadin, S., Pedrocchi, N. and Beschi, M. (2025) 'Reactive and safety-aware path replanning for collaborative applications', *IEEE Transactions on Automation Science and Engineering*, 22, pp. 21456-21468. doi: 10.1109/TASE.2025.3607037.

Welch, G. and Bishop, G. (1995) An introduction to the Kalman filter. Technical Report TR 95-041. Chapel Hill, NC: University of North Carolina at Chapel Hill.

Appendix: Data and Figure Provenance

This appendix records the submitted files used to produce the quantitative results and figures in Section 6. All paths are relative to the submitted project root. The final CSV and PNG outputs are stored in tests/result, and the generating scripts are included in tests.

Report result / figure	Submitted data and output files	Script provenance
Figure 6.1 benchmark setup visual	tests/result/figure_6_1_velocity_benchmark_setup.png	Captured from the GUI benchmark scene using tests/benchmark_visual_demo.py; used as a setup illustration rather than a quantitative result.
Figure 6.2.1 success rate vs obstacle speed	tests/result/benchmark_raw_trials_zero_2.csv tests/result/figure_6_2_1_success_vs_speed.png	tests/benchmark_raw_zero.py generates the raw trial CSV; tests/plot_results.py aggregates it and exports the figure.
Figure 6.2.2 safety- envelope stability	tests/result/benchmark_raw_trials_zero_2.csv tests/result/figure_6_2_2_stability.png	tests/plot_results.py uses the same velocity-sensitivity raw trial CSV to plot minimum-clearance stability.
Figure 6.2.3 Kalman noise robustness	tests/result/figure_6_2_3_kalman_noisy_observations.csv tests/result/figure_6_2_3_kalman_noisy_observations.png	tests/test_robustness.py generates the synthetic noisy trajectory, computes RMSE before/after filtering, and exports both CSV and PNG.
Figure 6.2.4 narrow- passage generalisation	tests/result/benchmark_report_raw.csv tests/result/benchmark_report_summary.csv tests/result/figure_6_2_4_narrow_passage_threepanel.png	tests/benchmark_runner.py produces raw and summary CSVs; tests/plot_section_6_2_4_generalisation.py plots the three-panel figure.
Figure 6.2.6 latency ablation	tests/result/ablation_continuous_metrics.csv tests/result/figure_6_2_6_ablation_min_dist.png	tests/benchmark_ablation.py records APF+KF and reactive APF metrics; tests/plot_ablation.py plots the mean minimum-distance comparison.
Strict zero- collision success summary	tests/result/benchmark_raw_trials_zero_2.csv tests/result/zero_collision_success.csv	tests/analyze_zero_collision.py derives the zero-collision summary from the velocity-sensitivity raw trial CSV.

The submitted source code and result files therefore provide a direct provenance trail from benchmark execution, through CSV aggregation, to the figures discussed in the report.